

A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint)' The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. (Sample data for reference is given below). Calculate current need of resources by each process. Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

CODE:



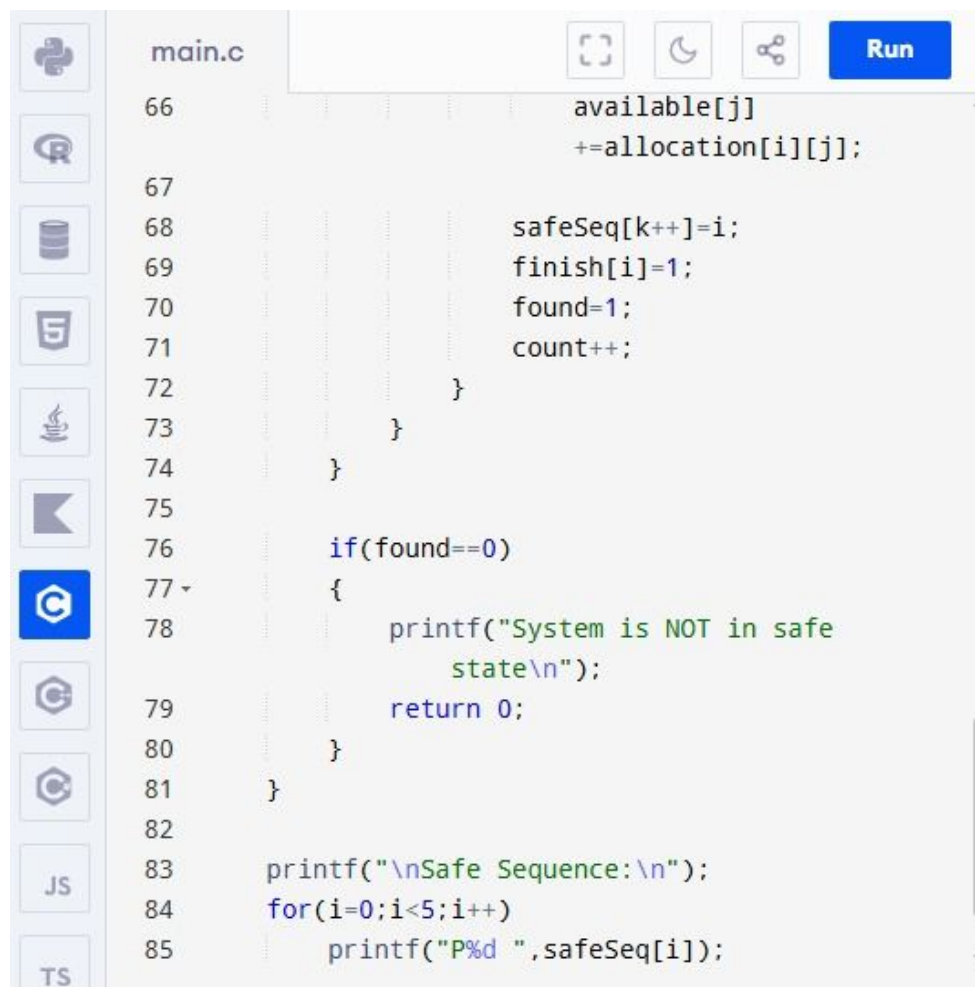
```
main.c
1  #include<stdio.h>
2
3  int main()
4  {
5      int allocation[5][4], max[5][4], need[5][4];
6      int available[4];
7      int finish[5]={0};
8      int safeSeq[5];
9      int i,j,k=0;
10
11     printf("Enter Allocation Matrix:\n");
12     for(i=0;i<5;i++)
13         for(j=0;j<4;j++)
14             scanf("%d",&allocation[i][j]);
15
16     printf("Enter Max Matrix:\n");
17     for(i=0;i<5;i++)
18         for(j=0;j<4;j++)
19             scanf("%d",&max[i][j]);
20
21     printf("Enter Available Resources:\n");
22     for(i=0;i<4;i++)
```

```
main.c

23     scanf("%d",&available[i]);
24
25     // Need calculation
26     for(i=0;i<5;i++)
27     {
28         for(j=0;j<4;j++)
29         {
30             need[i][j]=max[i][j]
                -allocation[i][j];
31         }
32     }
33
34     printf("\nNeed Matrix:\n");
35     for(i=0;i<5;i++)
36     {
37         for(j=0;j<4;j++)
38             printf("%d ",need[i][j]);
39         printf("\n");
40     }
41
42     int count=0;
```

```
main.c

44     while(count<5)
45     {
46         int found=0;
47
48         for(i=0;i<5;i++)
49         {
50             if(finish[i]==0)
51             {
52                 int flag=1;
53
54                 for(j=0;j<4;j++)
55                 {
56                     if(need[i][j]>available[j])
57                     {
58                         flag=0;
59                         break;
60                     }
61                 }
62
63                 if(flag==1)
64                 {
65                     for(j=0;j<4;j++)
```



```
main.c
66         available[j]
           +=allocation[i][j];
67
68         safeSeq[k++]=i;
69         finish[i]=1;
70         found=1;
71         count++;
72     }
73 }
74 }
75
76 if(found==0)
77 {
78     printf("System is NOT in safe
           state\n");
79     return 0;
80 }
81 }
82
83 printf("\nSafe Sequence:\n");
84 for(i=0;i<5;i++)
85     printf("P%d ",safeSeq[i]);
```

```
           state\n");
79         return 0;
80     }
81 }
82
83 printf("\nSafe Sequence:\n");
84 for(i=0;i<5;i++)
85     printf("P%d ",safeSeq[i]);
86
87 return 0;
88 }
```

Output :

```
Output Clear  
Enter Allocation Matrix:  
5*4)  
  
Enter Max Matrix:  
Enter Available Resources:  
  
Need Matrix:  
524283 0 731470553 31310  
2 0 6 -2147483648  
0 0 0 0  
0 0 0 0  
0 0 0 0  
System is NOT in safe state  
  
=== Code Execution Successful ===
```